

JAVASCRIPT ENGINE: Execute JavaScript programs

- Chrome - V8
- Firefox - SpiderMonkey
- Apple Safari - JavaScriptCore/Nitro
- Microsoft Edge - Chakra

How it works:

1. Parsing: Source code is parsed and converted into AST (Abstract Syntax Tree).
2. Bytecode Generation: The AST is used to generate bytecode (an intermediate representation).
3. Interpretation: The bytecode is executed line by line by the engine.
4. JIT Compilation: Frequently executed code/ hot code is optimized and compiled into machine code using Just-In-Time (JIT) compilation. This improves performance.
5. Execution: Synchronous code is executed immediately, and event loop manages the order of execution for async code. JavaScript engine has a call stack, two queues and a heap.

COMPONENTS:

1. Call stack: Executes synchronous function calls.
2. Microtask queue: High priority asynchronous task - Promises, then & catch, async & await, fetch (itself is synchronous but returns a promise whose resolution then or catch goes into microtasks).
3. Macrotask queue/Task queue: Low priority asynchronous task - Web API (Browser), setTimeout, setInterval, DOM events (click, scroll etc.), I/O events, setImmediate (Node.js).
4. Heap: Memory area where JavaScript stores dynamically allocated data. Stores objects, arrays, functions, closure, class instance etc. and is managed by garbage collector.

EVENT LOOP:

1. Execute all synchronous code first in the call stack until it's empty
2. Execute all microtasks (drain the microtask queue completely)
3. Execute only one macrotask - handled by browser/web API
4. After each macrotask, check and run all microtasks again before the next macrotask
5. Repeat

CODE:

```
console.log(1); // synchronous code

setTimeout(() => {
  console.log(2); // macrotask queue
}, 0);

Promise.resolve().then(() => {
  console.log(3); // microtask queue
});

console.log(4); // synchronous code
```

OUTPUT: 1 4 3 2

PARALLEL EXECUTION:

- JavaScript is single-threaded: only one piece of JavaScript runs at a time per runtime.
- Asynchronous operations are queued, and the event loop co-ordinates the callback execution.
- Async work itself (timers, network, I/O, crypto, etc.) is handled outside JavaScript's main thread by the browser/Node runtime or OS.

Browser: Web workers - allow true parallel execution of JavaScript.

- Each thread is its own JavaScript runtime having their own call stack, task queues (microtask & macrotask), heap and event loop.
- Communicates with main thread via message passing (postMessage).
- Cannot access DOM directly.

Node.js: Worker threads - provide parallel execution for CPU-heavy tasks.

- Each thread is its own JavaScript runtime having their own call stack, task queues (microtask & macrotask), heap and event loop.
- Libuv has 4 threads in thread pool by default.
- Communicates via message channels (parentPort, postMessage).
- Can share memory explicitly via SharedArrayBuffer.
- Node also uses libuv's thread pool for async I/O (file system, DNS, crypto), but those threads don't run JavaScript — they just handle background work.

TYPESCRIPT:

- **TypeScript** is an **open-source programming language** developed by **Microsoft** and its **superset of JavaScript**.
- It offers **static typing**, so we explicitly declare data type which helps catch errors **before running the code**.
- **Type inference** helps automatically determine data type if not declared.
- **Interfaces & Type alias** helps to define the structure of objects and ensure consistency.
- **Object-Oriented Programming Support**: Classes, inheritance, access modifiers (public, private, protected), abstract classes.
- **Compile-Time Error Checking**, not at runtime.
- Modern JavaScript Support (ES6+) and excellent tooling and IDE support, Intellisense.
- It is compiled into **(transpiled)** into plain JavaScript using typescript compiler which browsers and JavaScript engines understand.
- TypeScript configuration of a project is stored in tsconfig.json file. Compilation: .ts (TypeScript file) => tsc (tsc compiler) => .js (JavaScript file)
- Current version: v5.9 (Dec 2025)

REACT SERVER COMPONENTS:

- They are rendered on server and their output which is built up HTML is sent to the client.
- They can directly perform API calls or database queries since they run on the server.
- These functional components can be written as async functions, which allows them to fetch data during rendering. This is not the case in regular react functional components.
- This reduces client-side JavaScript since their code never runs in the browser. It makes the page load faster.
- Right now server components exist only in next js and in React 19 which are still in experimental stage.
- In next js, all components are server components by default meaning regular React useState, useEffect hooks will not work in them and require "use-client" directive to make them client components.
- Useful in SEO and the page content is already available before the page reaches browser.

REACT COMPILER:

- The React Compiler is a build-time optimization tool developed by **Meta** for **React** that automatically improves component performance.
- It analyzes your React code and applies optimizations like memoization without requiring manual hooks.

- It reduces the need for hooks such as `useMemo` and `useCallback` in many cases by handling them automatically.
- It prevents unnecessary re-renders by tracking how state and props are used within components.
- It works during the compilation (build) step, so there is no additional runtime overhead in the browser.
- It simplifies React development by automatically handling optimizations (like memoization), reducing bugs from dependency arrays and making code cleaner and easier to maintain.
- It improves performance by updating only the parts of the UI affected by data changes and is especially helpful in large applications—though it is still evolving and requires proper tooling support.

REACT HOCs:

- A Higher-Order Component (HOC) is a function that takes a component and returns a new enhanced component.
- It allows reuse of common logic across multiple components in React.
- HOCs are useful for handling things like authentication, data fetching, or logging.
- HOCs do not modify the original component; instead, they wrap it and pass additional props or behavior down to it.
- Overusing HOCs can make component trees harder to understand (wrapper nesting), which is why modern React development often prefers hooks for better readability.
- But they are used less often today in favor of hooks for simpler and cleaner code.

CLEAN ARCHITECTURE:

- Software design philosophy by Robert Martin (Uncle Bob).
- Separation of concerns: Build scalable, maintainable, testable applications.
- Dependency rule: Dependencies point inward (Dependency Inversion). Outer layers depend on inner layers, but inner layers should never know anything about outer layers. Loose coupling.
- Domain first, at the center.

LAYERS: Onion architecture (concentric circles)

1. Layer 1 - Frameworks & Drivers (outermost)
2. Layer 2 - Interface Adapters
3. Layer 3 - Use Cases (Application Business Rules)

4. Layer 4 - Entities - Enterprise Business Rules (innermost)

PROJECT FOLDER STRUCTURE: lib:

1. core: reusable utilities (network, errors, constants, routes, helpers).
2. features: feature-based modules (login, checkout, payments, etc.).

feature:

1. presentation: Bloc, Pages, Widgets
2. domain: Usecases, Entities, Repositories
3. data: Datasources, Models, Repository Implementations
4. feature-level dependency injection file

PRESENTATION LAYER:

1. Pages: screen-level UI.
2. Widgets: reusable UI components.
3. Bloc: Widgets dispatch events to Bloc and Bloc emits states back to widgets. event => bloc => state
 - Bloc calls usecase.
 - UI logic stays thin; business logic stays out of widgets.

DOMAIN LAYER:

1. UseCases: single responsibility classes with a call() method.
2. Repositories (abstract): define contracts, return Future<Either<Failure, Entity>>.
3. Entities: plain Dart classes representing business objects (no JSON, no framework code).
 - Independent of framework.
 - Key principle: Domain depends on nothing; everything depends on domain.
 - Repositories bridge Domain and Data layers

UseCase Pattern

- Input: Params or NoParams from Bloc.
- Output: Either<Failure, Entity>.
- Encapsulates one specific business action (e.g., GetUser, Login, FetchOrders).

DATA LAYER:

1. Repository (Implements repository contracts from domain): Convert Models <=> Entities. Handle Left(Failure) and Right(Data) responses.

2. Models: Extend Entities. Contain fromJson, toJson, and factory constructors.
3. Datasources: handle API calls, caching, database access.

Dependency Flow: Presentation => Domain <= Data

Data Flow: UI => Events => BLoC => Use Cases => Repository => Data Sources UI <= States
<= BLoC <= Entities <= Repository <= Data Sources

TDD (Test-Driven Development) Workflow:

1. Write tests before implementation.
2. Mirror lib/ structure inside test/.
3. Use Arrange => Act => Assert.
4. Mock dependencies with Mockito.
5. JSON fixtures provide predictable dummy data for tests.

Key Libraries:

1. get_it => dependency injection (service locator).
2. flutter_bloc => state management.
3. equatable => value equality for entities and states.
4. dartz => functional programming (Either, Left, Right).
5. http, data_connection_checker_nulls => remote API handling.
6. shared_preferences => local caching.

LAYERED ARCHITECTURE:

- Separation of concerns.
- Horizontal layers each with clear responsibility.
- Request and response pass from one layer to next.
- Typical 3-tier architecture: Presentation layer => Business / Domain Layer => Data Layer
- Request flow (top to bottom) => Client => Controller => Service => Repository => Database <= Response flow (bottom to top)

LAYERS:

1. Presentation Layer (Controller / API)
2. Application / Service / Domain Layer (Business Logic)
3. Data Access / Persistence / Infrastructure Layer (Repository / DAO)

4. Database

COMPONENTS

Controller:

- Handles HTTP request/response, input validation, routing.
- Serialization (Object => JSON) and Deserialization (JSON => Object) happens at the controller layer but its done by web framework (Express.js, Springboot etc.).
- Converted JSON is then sent to client.

Service:

- Contains business logic, orchestrates multiple repositories.

Repository:

- Contains DB queries, keeping db logic out of controllers or models.
- Fetches and persists data without exposing database details.
- Once we get data from DB, it is plugged in as per our model by repository and returned to service.
- Encapsulates all data access logic.

Model:

- Represents data or business concepts in a specific layer.
- Each layer has its own form of model. It represents same data but in different form at different layers.
- Presentation layer - DTO (Data Transfer Object)
- Domain layer (Entity / Business object)
- Data layer (Persistence model - ORM / ODM)

Controllers handle requests, Services decide what to do, Repositories handle data, and Models carry information between them.

Clean architecture vs Layered architecture:

- Layer arch. has flow from top to bottom, clean arch. has flow from outward to inward.
- Inner layers know nothing about outer layers.
- Business logic is fully independent.
- Main difference is dependency inversion.

- Presentation => Application (Use Cases) => Domain (Entities & Rules) <= Infrastructure (DB, Frameworks)

Layers:

1. Presentation Layer (Controller / API)
2. Application Layer (Service / Use Cases)
3. Domain Layer (Entities, Business Logic)
4. Data Access / Persistence / Infrastructure Layer (Repository / DAO)
5. Database

MODEL VIEW CONTROLLER:

- MVC is a design pattern that separates an application into Model, View, and Controller.
- The Model manages data, business rules, and application logic.
- Models are typically defined using Mongoose, which maps MongoDB documents to JavaScript objects.
- The Model handles database operations like CRUD, schema validation, and business logic.
- MongoDB's schema-less nature fits well with MVC, allowing flexible model evolution.
- The same Model can be reused for multiple Views, such as web pages and API responses.
- The View is responsible only for presenting data to the user.
- Views are built using HTML and CSS, often with templating engines like EJS or Handlebars.
- Client-side JavaScript enhances the View with dynamic behavior and DOM manipulation.
- The Controller handles user input and coordinates between the Model and View.
- Controllers receive requests from the frontend, process data, and return responses or rendered views.
- In a Node.js application, Express acts as the Controller layer, handling routes and HTTP requests.
- Middleware in Express integrates smoothly with MVC for authentication, logging, and validation.
- MVC helps keep routing logic separate from database queries, improving maintainability.
- Using MVC in Express makes APIs cleaner and more modular, especially for RESTful services.
- This separation improves code organization and readability.

- MVC makes applications easier to maintain and scale, making it easier to add new features or endpoints.
- Overall, MVC enables clean architecture, faster development, and easier collaboration.

SDLC Stages: Software Development Lifecycle

1. Planning
2. Requirement gathering & analysis
3. System Design - System architecture, DB design, HLD, LLD, UI/UX mockups
4. Implementation / Development
5. Testing
6. Deployment
7. Maintenance

STLC Stages: Software Testing Lifecycle

1. Requirement Analysis
2. Test Planning
3. Test Case Design / Development
4. Test Environment Setup
5. Test Execution
6. Test Cycle Closure

SDLC & STLC both run in parallel in agile project. Dev and QA both get requirements at same time and they start their respective work.

JAVASCRIPT:

- JavaScript is a **high-level, dynamically typed, interpreted, lightweight, single-threaded programming language** that follows the **ECMAScript specification**.
- It is mainly used for **building interactive and dynamic web applications** on the client side and for **developing backend APIs and server-side applications using Node.js**.
- Although JavaScript is an **interpreted language**, modern JavaScript engines (such as **V8**) use **Just-In-Time (JIT) compilation** to improve performance.
- The **latest ECMAScript version is ES15 (ECMAScript 2024)**.

HOISTING:

- **Hoisting** is a **JavaScript behavior** where **declarations of variables and functions** are **processed before code execution**. During the **memory allocation (creation phase)**, JavaScript **allocates memory for variables and functions** before **executing the code**.
- If a variable is declared **inside a function**, its **scope** is limited to that **function**, which means it **cannot be accessed outside** of that function. If it is **outside**, then it has a **global scope**; it can be **accessed anywhere**.
- **var** is **function scoped**, **let** and **const** are **block scoped**.
- **Only declarations** are **hoisted**, not initializations.
- **var variables** are **hoisted** and **initialized with undefined**.
- **let and const variables** are **hoisted but not initialized**.
- Hoisting **does work with let and const**, but they are placed in the **Temporal Dead Zone (TDZ)**. The **Temporal Dead Zone (TDZ)** is the time between the **start of a scope** and the **declaration of a let or const variable**, during which **accessing it throws a ReferenceError**. Accessing them **before their declaration** results in a **ReferenceError (not undefined)**.

Code snippets:

1. var

```
console.log(a);  
var a = 10;  
Output: undefined
```

2. let/const

```
console.log(b);  
let b = 20;  
Output: ReferenceError: Cannot access 'b' before initialization
```

3. Function declaration hoisting

```
sayHello();  
  
function sayHello() {  
    console.log("Hello");  
}  
Output: Hello
```

4. Function expression with var

```
sayHi();  
  
var sayHi = function () {  
    console.log("Hi");  
};  
Output: TypeError: sayHi is not a function
```

5. Block scope (let)

```
{  
    console.log(x);  
    let x = 5;  
}  
  
Output: ReferenceError
```

CLOSURE:

- **Function nesting:** Function is defined inside another function, and the inner function's scope is limited to the outer function unless returned or passed outside.
- **Closure:** When the inner function is returned or used outside the outer function, allowing it to remember and access the outer function's variables even after execution ends.

Code:

```
function outer() {  
  
  let count = 0;      // variable in lexical scope  
  
  return function inner() {  
  
    count++;          // inner remembers `count`  
  
    console.log(count);  
  
  };  
}  
  
const counter = outer(); // outer() has finished executing  
  
counter(); // 1  
  
counter(); // 2  
  
counter(); // 3
```

Working:

- outer() runs **once**, inner() is returned and stored in counter
- Even though outer() has finished, inner() still has access to count
- count persists between calls → **closure in action**

REACT:

- JavaScript library for building user interface.
- React maintains a **lightweight copy** of the actual DOM in memory called the **virtual DOM**.
- When state or props of UI components change, React creates a new virtual DOM and compares it with the old DOM using a **diffing algorithm** to identify changes.
- Then react updates only the changed parts of the real DOM. This process of comparing and updating the real DOM based on these changes is called **reconciliation**.
- This **minimizes re-rendering**. So, when there are frequent DOM manipulations they happen in memory and only minimal updates are applied to the browser's actual DOM.
- Latest version: v19.2 (Dec 2025)

FEATURES:

- Virtual DOM
- JSX - It stands for JavaScript XML is a HTML like syntax which can be used to do JavaScript programming.
- Reusable components
- Data flow: It has one way data flow which is from parent component to child components via props. Passing data between components - lifting state up and prop drilling.
- Hooks: React hooks is another major functionality
- Server components: It provides some server components and can be used for server-side rendering.
- Server-side rendering: It's a process in which the page renders on the server, and the HTML code is sent to the client. It decreases the loading time of the page.
- Compatibility: It can be easily use with other technologies like Express, Node for MERN stack
- SEO: It can help increase the website ranking for SEO.

CORE CONCEPTS:

1. State:

- State is the data which a react component holds.
- Local state: useState hook, Global state: state management libraries like redux.
- State variables must not be mutated or updated directly. First, create a copy and then modify it.

- Usually, the state variable of a parent component acts as a prop for a child component.
- 2. Props:**
 - Props are the attributes or arguments passed to a react component.
 - Props are read only; their value must not be changed.
 - 3. Hooks:**
 - Hooks are the inbuilt functions in react which we can use for accessing and modifying the state variable or doing various operations throughout a component's lifecycle.
 - 4. Virtual DOM & Reconciliation:**
 - React uses a virtual DOM to compare changes in memory and updates only the necessary parts of the real DOM through reconciliation, minimizing re-renders and improving performance.
 - 5. Webpack bundler:**
 - Bundler is a tool which takes all the JavaScript files and all the code in the project and creates single individual HTML, CSS, js files in the form of static assets which the browser uses to display the website.
 - Webpack is a type of bundler which react uses.
 - 6. Babel:**
 - Babel is a JavaScript compiler which transforms or compiles the JavaScript code in such a version that all the browsers can understand it. It makes advanced JavaScript code compatible with older browsers.

REACT 19 features:

- **Actions:** Built-in **async functions** for **forms, mutations, loading, errors, and optimistic updates** — no extra state boilerplate.
- **useActionState Hook:** Handles **async state + results** together, ideal for **form submissions**.
- **useOptimistic Hook:** Enables **instant UI updates** with automatic **rollback on failure**.
- **React Compiler:** Automatically **optimizes re-renders** by reducing the need for **useMemo, useCallback, and memoization** in most cases.
- **Server Components:** Seamless server-side rendering with client integration for faster apps.
- **Server Actions:** Handle mutations directly from server, with auto *pending* and *error* states.
- **Direct Ref Passing:** Pass **ref as a normal prop**, reducing **forwardRef** usage.
- **Document Metadata in Components:** Define **<title>, <meta>, <link>** inside components — React manages them safely.

NEXT JS:

- Next JS is a **react based full stack framework** developed by **Vercel** used to build **modern scalable applications**.
- It has lot of **powerful features** and is sometimes better than react in terms of performance, it has built-in routing, server-side rendering and API handling.
- Supports **hybrid rendering** like **SSR**, **SSG**, and **ISR** for flexibility.
- It uses the **App Router** with **file-based routing** for **simple navigation**.
- Enables **Server Components** to improve performance and reduce bundle size.
- Provides **built-in API Routes** to create backend endpoints easily.
- Provides **font and image optimization** through built-in components.
- Includes **automatic code splitting** for faster page loads.
- **Middleware** – Run logic before page load (auth, redirects).
- Current version - (Dec 2025) Next js 15

RENDERING METHODS:

CSR: Client-Side Rendering

- **Browser** does the **rendering** - the server sends a basic HTML shell and a **JavaScript bundle**.
- The browser downloads the JS, fetches data, and **builds the UI dynamically**.
- This shifts the work to the **client**, so **the initial load** can be slower.
- Once rendered, JavaScript handles all interactions and navigation.
- Good for SPAs but bad for SEO because the page does not have content as content is loaded in the browser after the user visits the website.

SSR: Server-Side Rendering

- **Server renders the full HTML** page for **every request** and sends it to the browser.
- User **makes requests**, **server fetches data**, and **renders** React components to HTML.
- Complete HTML arrives at the browser ready to display, improving **first paint**.
- Then **hydration** happens which means adding interactivity and event listeners to static HTML.
- Server workload is higher because rendering happens **per request**.
- Great for **SEO-heavy** pages like **blogs**, **marketing sites**, or **product pages**.

SSG: Static Site Generation

- HTML pages are **pre-generated** at build time and served as static files.
- Build process **generates every page upfront** so it can be very long, but the website is faster on the client.
- **CDN** serves pre-built HTML instantly when users request pages (fastest delivery).
- Pages are static, and if something is updated, **a manual rebuild** is required to see changes.
- Best for: blogs, documentation, marketing websites where content does not change often, and hence frequent builds are not required.

ISR: Incremental Static Regeneration

- Combines static generation with **on-demand regeneration** after **specified time intervals**.
- Initial build creates static pages, serves them like SSG.
- After revalidation period **expires**, the next request **triggers background regeneration**.
- Users get stale page until **fresh version builds and replaces it**.
- Best for: e-commerce with product catalogs, news sites needing freshness without constant rebuilds.

BDD:

- **Behavior Driven Development:** Software development approach where we **define and test the expected behavior** of application using **simple, natural language** that all stakeholders can understand.
- The **end-to-end test scenarios** are written in **feature files**. Uses Cucumber and playwright together.
- **Playwright:** A **Modern end-to-end testing framework** developed by **Microsoft** for **automating web browsers and testing web applications**.
- Playwright supports **cross-browser testing** (Chromium, Firefox, WebKit) and **parallel test execution**.
- **Cucumber:** A **BDD testing framework** that allows users to write test scenarios in plain, english language.
- Cucumber uses **Gherkin language** which has **Given-When-Then syntax** so that non-technical stakeholders can understand the test scenarios.
- Scenarios have **step definitions** which map to actual test code in any given language such as Java, JavaScript, or Python, etc.
- Steps written in Cucumber are **executed through the step definitions**, and **those step definitions call Playwright code** to automate the browser.

UNIT TESTING:

- React Testing Library: Lightweight package used to test react components.
- Jest: A JavaScript testing framework used with RTL.
- Jest runs tests in a **jsdom environment**, simulating a browser-like DOM for component rendering.

Internationalization (i18n): Building a software product in such a way that it can support **multiple languages, regions, time-zones and formats** without code changes.

Localization (l10n): The act of **adapting an internationalized UI** for a specific **locale or region** by providing translated text and region-specific formatting. Involves loading **locale-specific resources** (strings, formats, assets) and adjusting UI for **cultural and linguistic expectations**.

- Text Translation (UI text, error messages, etc.)
- Plurals & Gendered Translations
- Date & Time Formatting (formats, time zones)
- Number & Currency Formatting (decimal separators, currency symbols)
- Phone Number Formatting (country codes, number formats)
- Address Formatting (postal codes, street names)
- Cultural Sensitivity & Adaptation (colors, images, symbols)
- UI Directionality (RTL/LTR) (for languages like Arabic or Hebrew)
- Legal & Regulatory Information (terms of service, privacy policies)
- Measurements & Units (temperature, distance, weight)

Accessibility (a11y): Design and development of digital products for people with physical disability and visual/audio/sensory impairments.

- Semantic HTML (proper use of HTML tags like <header>, <main>, etc.)
- ARIA Roles & Attributes (e.g., aria-label, aria-live, role="button")
- Keyboard Navigation (focus management, keyboard events)
- Color Contrast (sufficient contrast between text and background)
- Form Accessibility (proper labels, inputs, error messages)

Physical disabilities:

- Blind – Screen reader
- Low vision – Screen magnifier
- Color blindness – Text labels/pattern for colored content

- Hearing impairment – Captions, subtitles, transcripts, visual cues (like alert, notification, warning)
- Motor disability/Cognitive disability – Assistive technologies – Keyboard navigation, joystick, large touch area, simple and clear instructions, simple navigation etc.
- Keyboard navigation – Using enter key to submit form instead of clicking on submit, using tab keys to navigate between fields.

STEPS POST DEVELOPMENT:

1. **Linting:** Enforce consistent code style, catch syntax and logic errors. Tool – ESLint
2. **Formatting:** Automatically format code for consistency. Tool – Prettier
3. **Transpile:** Used to convert modern JavaScript code into backward compatible javascript for old browsers. Transform + Compile. Tool - Babel. File for Babel configuration: .babelrc or babel.config.js
4. **Bundling:** Combine multiple JS files/modules into a single or few bundles for efficient loading. Tool - Webpack. File for webpack configuration: webpack.config.js
5. **Optimization:**
 - **Minification:** Reduce file size by removing whitespace, comments, and shortening variable names. Tool – Terser
 - **Tree Shaking:** Remove unused code from bundles.
 - **Code splitting:** Splitting large bundle of javascript code into smaller chunks which are loaded on demand. Eg: import syntax, lazy and suspense.
 - **Dead code removal:** Remove unreachable code segments.
 - **Asset optimization:** Optimize images, fonts, and other static assets.
 - **Caching & hashing:** Add content hashes to filenames for long-term caching.
6. **Building:** Output final production-ready files into a folder, typically named `build/` or `dist/`
7. **Deployment:** Deploy on a server. Static hosting, traditional server, cloud platform, container.

PERFORMANCE:

- **Reduce JavaScript Cost:** Minimize the amount of JavaScript downloaded, parsed, and executed to improve load and interaction times.
- **Lazy Loading:** Load code, images, or components only when needed to reduce initial page load time.
- **Optimize Assets:** Compress and optimize images, CSS, and JavaScript to reduce network payload size.
- **Improve Loading Strategy:** Prevent render blocking by deferring scripts and prioritizing critical resources.

- **Caching:** Store previously fetched resources, so repeat visits avoid unnecessary network requests.
- **Debouncing & Throttling:** Control how frequently expensive event handlers execute to reduce main-thread workload.
- **Memoization:** Cache results of expensive computations or renders to avoid unnecessary recalculation.
- **Reduce Network Latency:** Use CDNs and modern protocols to minimize request and response times.
- **Optimize Rendering:** Reduce reflows and repaints by managing DOM size and using efficient CSS properties.
- **Improve Runtime Performance:** Offload heavy tasks from the main thread and optimize framework re-renders.
- **Measure & Monitor Performance:** Track metrics like Core Web Vitals to identify bottlenecks and validate improvements.

NODE:

- JavaScript **runtime environment** which runs on **Google chrome's V8 javascript engine**.
- Executes **outside** the browser.
- **Single threaded**: It executes one main thread sequentially, so the code runs line by line.
- **Event loop**: Async tasks are handled by the **event loop**, so the main thread never blocks.
- Has **non-blocking I/O**: Uses **libuv** library to perform I/O operations.
- **Worker threads**: For CPU-heavy tasks, Node.js supports **worker threads** to run JavaScript in **parallel** from a thread pool.
- **Event driven**: Supports **asynchronous code** execution - Uses promises, callbacks and async/await.
- It has **built-in modules** for various functionalities - like file handling, encryption, data fetching, etc.
- Modules are written in JavaScript & C++ language
- Cross platform, Rich ecosystem, Scalable, real-time applications
- Free, open source
- Current version: v24.x (Dec 2025)

EXPRESS:

- Fast, lightweight, minimalist framework
- Building **web applications** and **APIs** with Node js
- Supports **middleware** to intercept and process requests and responses
- `express.json()` for JSON and `express.static()` for serving static files
- Supports **routing**
- Supports templating engines
- RESTful API Development
- Static file serving
- Unopinionated - so we don't have to follow a fixed project structure
- Current version: v5.x (Dec 2025)

Mongo DB:

- MongoDB is a **distributed, document-oriented NoSQL** database, storing data in **JSON-like documents**.

- It internally stores data in **Binary JSON (BSON)**, a binary-encoded format that is faster to parse and supports more data types than JSON.
- It uses a **flexible, schema-less design** so that we can use dynamic data structures. Although we can put **schema validation** using **Mongoose**.
- **High availability** - achieved through replica sets that provide automatic failover and redundancy.
- **Horizontal scaling** - done using sharding to distribute data across multiple servers for large workloads.
- **Aggregation pipeline** - a powerful framework to transform, filter, and analyze data in stages.
- **ACID transactions** - supported since MongoDB 4.0 for multi-document operations, ensuring atomicity, consistency, isolation, and durability.
- **Rich indexing** - supports text, geospatial, compound, and other indexes for faster query performance.
- Current version: v7.x (Dec 2025)

BACKEND PERFORMANCE:

- **Query optimization** involves using proper **indexes**, avoiding **full collection scans**, selecting only required fields (**projection**), and writing **efficient MongoDB queries** to reduce response time.
- Implement **caching** (e.g., **Redis**) for frequently accessed data.
- Apply **pagination, filtering, and sorting** to avoid large responses.
- Keep APIs **asynchronous and non-blocking** using **async/await** in Node.js.
- Reduce response size with **data projection** and **gzip compression**.
- Minimize **redundant database calls** by reusing cached or computed data.
- Use **connection pooling** for databases like **MongoDB**.
- Implement **rate limiting** and **throttling** to protect the API.
- Optimize **express middleware order** to reduce unnecessary processing.
- Continuously **monitor and profile performance** using **logging, PM2, or APM tools**.

BACKEND SECURITY:

- Use **authentication** mechanisms like **JWT** or **OAuth** to verify users.
- Implement **authorization** with **role-based access control (RBAC)**.
- Validate and sanitize all inputs to prevent **SQL/NoSQL injection** attacks.

- Secure sensitive data using **encryption** and **password hashing** (e.g., **bcrypt**).
- Apply **rate limiting** and **throttling** to prevent **brute-force attacks**.
- Use **HTTPS / TLS** to encrypt data in transit.
- Configure **CORS** properly to control cross-origin requests.
- Store secrets in **environment variables**, not in source code.
- Use **security headers** (via **Helmet** in Express) to protect against common attacks.
- Log and monitor suspicious activity using **logging and alerting tools**.

BACKEND CONCEPTS:

- **Rate limiting:** A way to control the number of **requests** a **client** (like a **user, app or system**) can make to a **server** within a specific **period**.
- **Load balancing:** A way to distribute **incoming requests or traffic** across multiple **servers** to ensure no single server gets **overloaded**.
- **Caching:** A way to store **frequently accessed data** temporarily so that it can be retrieved **quickly** without repeatedly fetching it from **the original source**.
- **CDN:** A **network of servers** distributed across different **locations** that work together to deliver **content** to users more **quickly**.
- **Microservices:** An **architectural style** where an application is divided into **smaller, independent services**, each responsible for a specific piece of **functionality**.
- **API Gateway:** A **server** that acts as an **entry point** for all **client requests** to your system. It **routes** those requests to the appropriate **microservice**, handles **load balancing** and other tasks.
- **Webhooks:** A way for one **application** to send **real-time updates or notifications** to another application when a specific **event** happens.
- **Sharding:** Method of splitting large **database** into smaller, more manageable pieces called **shards**.
- **Proxy:** **Server** that acts as an **intermediary** between **client** (like browser) and another **server** (like a website).
- **Message Queues:** **System** used to send, store and retrieve **messages** between different parts of an **application** (or between applications) in a **reliable and organized way**.
- **Backend** consists of a **physical server hardware instance** and **server software** like **nginx, Apache, tomcat, express, Django, IIS** etc.
- **Contract API / Mock API**
- **API Design doc**

DATABASE CONCEPTS:

- **MongoDB:** A NoSQL, document-oriented database designed for **scalability and flexibility**. It stores data as **BSON documents** instead of rows and tables.
- **Document:** A JSON-like structure containing **key-value pairs** with flexible fields. It is the **basic unit of data** in MongoDB.
- **Collection:** A group of **documents** stored together without a fixed schema. It is equivalent to a **table** in relational databases.
- **Database:** A logical container that holds **collections**. Multiple databases can exist on a **single MongoDB server**.
- **BSON: Binary JSON** used internally by MongoDB for storage and queries. It supports **additional data types** and improves **performance**.
- **BSON data types:** Includes **ObjectId, Date, Decimal128, Binary**. These types allow **accurate representation** of real-world data.
- **Schema-less design:** MongoDB does not enforce a schema at the database level. However, **schema design is handled at the application level**.
- **Embedding:** Related data is stored in the **same document**. This improves **read performance** and reduces joins.
- **Referencing:** Related data is stored in **separate documents** using references. This avoids **data duplication** and is useful for large datasets.
- **ObjectId:** Default value for the **_id** field that uniquely identifies a document. It contains **timestamp and machine information**.
- **ObjectId ordering:** ObjectIds are **roughly time-ordered**. This helps in **sorting and pagination**.
- **Insert operations:** `insertOne()` and `insertMany()` are used to add documents. If **_id** is missing, MongoDB **auto-generates it**.
- **Read operations:** `find()` returns multiple documents, `findOne()` returns one. Queries use **filters and projection**.
- **Update operations:** `updateOne()` and `updateMany()` modify documents. Update operators like **\$set, \$inc** are commonly used.
- **Delete operations:** `deleteOne()` and `deleteMany()` remove documents. Deletes are based on **filter conditions**.
- **Query operators:** **\$gt, \$lt, \$gte, \$lte** filter numeric and date values. They help in **range-based queries**.
- **Logical operators:** **\$and, \$or, \$not** combine multiple query conditions. Used for **complex filtering logic**.
- **Array operators:** **\$in, \$all, \$elemMatch** query array fields. They allow **matching multiple or nested values**.

- **Index:** A data structure that **speeds up read queries**. Indexes increase **storage usage and write cost**.
- **Compound index:** An index created on **multiple fields**. Field order matters for **query optimization**.
- **Multikey index:** Automatically created when indexing **array fields**. Enables fast searches inside arrays.
- **Aggregation framework:** Used for **data processing and analytics**. It works using a **pipeline of stages**.
- **Aggregation pipeline:** Each stage processes documents and passes results forward. Common stages include **\$match, \$group, \$sort, \$project**.
- **Replication:** Ensures **high availability and data redundancy**. Data is copied across multiple servers.
- **Replica set:** A group of MongoDB instances with **one primary**. Automatic **failover** occurs if the primary fails.
- **Sharding:** Distributes data across **multiple machines**. Used for **horizontal scaling** of large datasets.
- **Shard key:** Field used to **split and distribute data**. A poor shard key can cause **performance issues**.
- **Atomicity:** MongoDB guarantees atomic operations at the **document level**. Either the whole document is updated or not at all.
- **Transactions:** Support **multi-document ACID operations**. Useful for scenarios requiring **strong consistency**.
- **Write concern:** Defines **how many nodes must acknowledge a write**. Higher levels increase **data safety**.
- **Read concern:** Controls **how consistent the data returned is**. Higher read concern means **more reliable reads**.
- **TTL index:** Automatically deletes documents after a **specified time**. Commonly used for **logs and session data**.
- **Capped collection:** Fixed-size collection that maintains **insertion order**. Old documents are removed when the size limit is reached.
- **GridFS:** Used to store **large files** like videos or images. Files are split into **chunks and stored across collections**.

FLUTTER:

- Flutter is an **open-source SDK** developed by **Google**.
- Used to build **natively compiled** applications (compiled to **ARM, x86** and **JavaScript** for the web).
- Used to develop **multi-platform applications**: Mostly mobile applications but also web, desktop and embedded applications.
- It uses **Dart programming language**, and a **single code base** is used for developing applications.
- It allows for **fast development** due to **hot reload** and **hot restart** features.
- It offers **high performance** due to its **Impeller rendering engine** and **ahead of time compilation**.
- Provides rich and beautiful widgets for UI - **Material Design widgets** (Android-style) and **Cupertino widgets** (iOS-style).
- It has a **rich ecosystem** where **pub.dev** provides lots of **plugins and packages**.
- Current version: v3.x (Dec 2025).

DART:

- **Dart** is an **open source, object-oriented programming language** created by **Google**.
- It's used to build **fast, scalable apps** for **multiple platforms** like mobile, desktop, and web. Mainly used in **Flutter**.
- Syntax is like **C++, C#, and Java**.
- **Strongly typed**, with **type inference** (**var**), and optional dynamic typing.
- **Null safety** — eliminates null reference errors with compile-time checks.
- **Asynchronous programming** using: **async/await, Futures, Streams**
- Data structures: Dart supports **List, Set, Map, Queue**, along with **Iterable** for collection handling. These are used to store and manage grouped data efficiently.
- Dart OOP uses **class** to create **objects** with **constructors, variables, and methods**, supports **inheritance (extends)**, **abstraction (abstract / interface)**, **polymorphism (@override)**, and **encapsulation**, where **final** means assign once, **@required** enforces mandatory parameters, **@override** ensures correct method overriding, and **super** accesses parent class members.
- **Garbage collected**, memory-managed language.
- Supports **JIT compilation** during **development**. So, dart code is compiled on the fly using Dart VM making the hot reload possible.
- Supports **AOT compilation** in **production build** to compile native machine code for maximum performance.

- Current version: v3.x (Dec 2025).

Flutter Commands:

- **flutter create** <project_name> – Creates a new Flutter project
- **flutter pub add** <package_name> – Adds a new dependency
- **flutter pub get** – Gets all dependencies from pubspec.yaml
- **flutter pub upgrade** – Upgrades all dependencies
- **flutter pub outdated** – Shows outdated dependencies
- **flutter doctor** – Checks Flutter setup and environment
- **flutter run** – Runs the app on a connected device or emulator
- **flutter run -d** <device_id> – Runs the app on a specific device
- **flutter clean** – Cleans the build cache
- **flutter devices** – Lists connected devices
- **flutter emulators** – Lists available emulators
- **flutter build apk | ios | web** - Builds the app for release for the specified platform.

Flutter Widgets:

- `main => runApp => MyApp => return MaterialApp => home => Homepage widget OR
main => runApp => return MaterialApp => home => Homepage widget (lean - one less
MyApp widget)`
- `Page widget => Scaffold => appBar, body`

Stateless widgets: A widget that **does not change its state** during its lifetime. Its UI depends only on the input data.

Stateful widgets: A widget that **can change its state** during runtime and rebuild its UI when the state changes.

BLoC Pattern:

- **BLoC (flutter_bloc)** is a **state management library** that separates **UI (presentation)** from **business logic**, following the flow **event → bloc → state** and clean architecture principles.
- **Events** represent **user interactions** (button click, scroll, input). UI dispatches events (e.g., `GetListData`, `LoadMoreData`) to the Bloc along with required request parameters.
- **States** represent the **UI condition** (Initial, Loading, Loaded, Error). When Bloc emits a new state, the UI automatically re-renders based on that state.
- **Bloc** acts as the middle layer: it receives events, calls **usecases**, processes results using **Either + fold**, and **yields states** using `async*` and `yield`.

- **flutter_bloc widgets** (BlocProvider, BlocBuilder, BlocListener, MultiBlocProvider, etc.) are used to **provide blocs**, **rebuild UI on state changes**, and **perform side effects** like dialogs or snackbars.

CONTAINERS:

- **Lightweight, standalone, portable and executable software packages** that include everything needed to run a piece of software, such as the code, runtime, libraries, dependencies, and system tools.
- They ensure consistency and reproducibility across different environments.
- Application and its dependencies are packaged as a single unit called a container and deployed on a container platform. It makes them reusable.
- Multiple packages can be deployed on a container platform, and all of them use the same OS and same hardware. Unlike VM, which has a guest OS.

MICROSERVICES:

- A **software architecture pattern** where application is divided into **multiple services**. Each service is usually run in a **separate container** and responsible for **single functionality**. Eg: **User Authentication, Payments, Catalog**, etc.
- Services are **loosely coupled, developed and deployed by different teams**, use **different tech stacks independent of each other**, and services **communicate via APIs**.
- Making **changes to one component** does **not break other components**. Also, the **complete application does not need to be redeployed**.

DOCKER:

- **Docker** is an **open-source platform** which is used to **build, package, and run applications** in containers.
- **Docker images: Read-only templates** used to create **containers**. They include the **application code, libraries, and dependencies**. Images can be built from a **Dockerfile**, which is a **script** that contains a series of **instructions** on how to create the image.
- **Docker image vs Docker container: Image** is just the **artifact**. **Container** is the **runtime environment** for image.
- **Docker file: Text file** with a set of **instructions** to build our own **Docker image**. **Blueprint** of the docker image. Each **instruction** in a **Dockerfile** creates a **layer** in the image. Dockerfiles enable **automation** of the image creation process, making it easy to **reproduce and share** container configurations.
- **Docker Hub: Default public registry**. Docker Hub is a **cloud-based registry service** where Docker users can **store and share** their container images. It provides a **centralized resource** for **finding and distributing** containerized applications.
- **Private registries/repository** can also be set up for **secure and controlled distribution** of images within an **organization**.

DOCKER COMMANDS:

```
docker run -d -p 6001:6379 --name redis-new redis:4.0
```

-p host port: container port

- -d: Run in detached mode (in the background).
- -p 6001:6379: Map port 6379 inside the container to port 6001 on the host machine.
- --name redis-new: Name the container "redis-new".
- redis:4.0: Use the Redis image, specifically version 4.0.

```
docker run -p 27017:27017 -d -e MONGO_INITDB_ROOT_USERNAME=admin -e  
MONGO_INITDB_ROOT_PASSWORD=password --name mongo --net mongo-network
```

- -p 27017:27017: Map port 27017 inside the container to port 27017 on the host machine (default MongoDB port).
- -d: Run the container in detached mode.
- -e MONGO_INITDB_ROOT_USERNAME=admin: Set the environment variable MONGO_INITDB_ROOT_USERNAME to "admin" (initial root username for MongoDB).
- -e MONGO_INITDB_ROOT_PASSWORD=password: Set the environment variable MONGO_INITDB_ROOT_PASSWORD to "password" (initial root password for MongoDB).
- --name mongo: Name the container "mongo".
- --net mongo-network: Connect the container to a user-defined network named mongo-network. This ensures that the MongoDB container can communicate with other containers on the same network.

OTHER

- docker ps: list all running containers
- docker ps -a: list running and stopped containers
- docker run image_name: start an image - docker pull + docker start. Create and start a new container from an image.
- docker start containerID: start stopped container
- docker start container1ID container2ID container3ID: start multiple containers
- docker stop containerID: stop a container
- docker stop container1ID container2ID container3ID: stop multiple containers
- docker restart containerID or containerName: Restart one or more containers
- docker kill containerID or containerName: Kill one or more running containers
- docker rm containerID or containerName: Remove one or more containers
- docker images: check existing images in registry
- docker pull image_name: Pull an image from a registry

- `docker push image_name`: Push an image to a registry
- `docker build -t image_name`: Build an image from a Dockerfile
- `docker rmi image_name`: Remove one or more images
- `docker tag image_name new_image_name`: Create a tag for an image

KUBERNETES:

- Kubernetes (K8s) is an **open-source container orchestration** platform that **automates the deployment, management, scaling, and load balancing of containerized applications**.
- **Pods**: Pods are the **smallest deployable units** in **Kubernetes**, representing a **single instance** of a **running process** in a **cluster**. Each pod can contain **one or more containers** that share the same **network namespace** and **storage**.
- **Nodes**: Nodes are the **physical or virtual machines** that make up a **Kubernetes cluster**. Each node runs a **container at runtime** (like **Docker**), along with the **Kubernetes node agent (kubelet)** and other **necessary components**.
- **Clusters**: A cluster is a **set of nodes** that work together to run **containerized applications** managed by **Kubernetes**. It includes at least one **master node** for **control plane activities** and several **worker nodes** for running **application workloads**.
- **Services** provide a **stable network endpoint** for accessing a **set of pods**. They **abstract the underlying pods** and enable **communication between components** without needing to track **pod IP addresses** directly.

Main kubectl Commands - K8s CLI:

Layers of abstraction: Deployment manages replicaset => Replicaset manages POD => POD is abstraction of => Container

- `kubectl get nodes | pod | services | replicaset | deployment`: Status of different K8s components
- `kubectl get pod -o wide`: Detailed status of PODs
- `kubectl create deployment [deployment-name] --image [docker-image]`: Create deployment
- `kubectl describe pod [pod-name]`: Get POD info
- `kubectl exec -it [pod-name] -- bin/bash`: Get interactive terminal
- `kubectl create -f <file.yaml>`: Creates resources defined in a YAML or JSON file.
- `kubectl apply -f <file.yaml>`: Applies changes defined in a YAML or JSON file to existing resources.
- `kubectl logs <pod-name>`: Displays the logs of a specific pod.
- `kubectl logs <pod-name> -c <container-name>`: Displays the logs of a specific container within a pod.

JENKINS:

- Jenkins is an **open-source automation server** which is used to **automate various stages** in a **CI/CD pipeline** such as **build, test, and deploying applications**.
- It comes with a lot of customizable plugins useful for automating repetitive tasks, making processes faster.

DevOps lifecycle stages:

1. **Plan** – Define requirements and roadmap
2. **Code** – Develop application source code
3. **Build** – Compile and package the code
4. **Test** – Validate functionality and quality
5. **Release** – Approve build for deployment
6. **Deploy** – Deliver application to production
7. **Operate** – Run and maintain the system
8. **Monitor** – Track performance and gather feedback

Continuous integration: 1 – 4: Plan, Code, Build, Test

Continuous delivery/deployment: 5 – 8: Release, Deploy, Operate, Monitor

Key Jenkins Terms

1. **Pipeline** – Suite of plugins that support continuous delivery using Groovy-based DSL.
2. **Node** – Machines (master/agents) that run tasks.
3. **Job/Project** – Defines tasks like build/test/deploy.
4. **Build** – One execution of a job.
5. **Plugin** – Extends Jenkins functionality.
6. **Freestyle Project** – Simple non-code-based job configuration.
7. **Declarative Pipeline** – Simple structured syntax for pipelines.
8. **Scripted Pipeline** – Advanced Groovy-based pipeline scripting.
9. **Workspace** – Directory where job executes and stores files.
10. **Artifact** – Output files produced by the build.
11. **View** – Organizes jobs visually.
12. **Credential** – Securely stored secrets (username/password/token).

Jenkins File:

- A **Jenkins file** is a text file that defines a Jenkins **CI/CD pipeline as code**.
- It describes **how Jenkins should build, test, and deploy** an application.
- Written in **Groovy** using **Declarative** or **Scripted** pipeline syntax.
- Enables **versioned, repeatable, and auditable** CI/CD pipelines.

- Automatically executed by Jenkins when triggered (e.g., on code changes).

CORE CLOUD & AWS FUNDAMENTALS:

1. AWS Global Infrastructure:
 - **Regions, AZs, Data Centers:** AWS deploys compute/storage across **isolated Regions** made of multiple **Availability Zones (AZs)** to ensure **high availability** and **fault isolation**.
 - **Edge Locations & PoPs:** Global network of **CDN endpoints** used by **CloudFront** and **Route 53** to provide **low latency** and faster content delivery.
2. AWS Interfaces & Tools
 - **AWS Management Console:** Web-based UI to manage AWS resources with **point-and-click operations** and account-level visibility.
 - **AWS CLI:** Command-line tool to automate AWS actions via **scripts**, useful for DevOps and repeatable workflows.
 - **AWS SDKs:** Language-specific libraries (Python, Java, Node.js, etc.) enable applications to call **AWS APIs programmatically**.
 - **AWS Cloud Shell:** Browser-based **pre-configured CLI environment** with credentials auto-loaded for immediate AWS command usage.
 - **EC2 Instance Connect:** Secure method to **SSH into EC2** using **temporary keys**, reducing the need to manage static key pairs.
3. Identity & Access Management
 - **IAM (Policies, Roles, Groups, MFA):** Central service to control **who can access** **JSON policies**, **temporary roles**, user **groups**, and **multi-factor authentication**.
 - **AWS STS:** Issues **temporary security credentials** for roles, cross-account access, and short-lived authenticated sessions.
 - **AWS Directory Services (AD, AD Connector, Simple AD):** Managed **Active Directory** options for integrating AWS resources with **Windows authentication**.
 - **Amazon Cognito:** Provides **user sign-up, sign-in, and token-based auth** for apps, with support for **OAuth, SAML, and social logins**.

COMPUTE / VM:

- **EC2 — Elastic Compute Cloud:** Provides **virtual servers** with customizable CPU, memory, storage, and networking options. Commonly used for hosting applications, backend services, and compute-intensive workloads.
- **EBS — Elastic Block Store:** Durable **block-level storage** for EC2 with AZ-level replication for high availability. Supports **snapshots, encryption**, and various performance tiers.
- **AMI — Amazon Machine Image:** A **prebuilt template** containing OS, configuration, and software packages. Ensures consistent and repeatable EC2 deployments across environments.

- **EC2 Instance Store: High-speed ephemeral storage** directly attached to the physical host. Ideal for temporary data like caches, buffers, or batch-processing scratch space.
- **EFS — Elastic File System:** Fully managed, scalable **NFS file system** accessible by multiple EC2 instances. Automatically grows and shrinks with your data, offering multi-AZ durability.
- **ELB — Elastic Load Balancer:** Distributes incoming traffic across multiple instances to improve **availability** and **application resilience**. Supports health checks and automatic failovers.
- **CLB — Classic Load Balancer:** Legacy **Layer 4/7 balancer** used for basic HTTP/HTTPS and TCP load distribution. Suitable for simple, older architectures.
- **ALB — Application Load Balancer: Layer 7 load balancer** enabling path/host routing, WebSocket support, and advanced HTTP features. Ideal for microservices and containerized apps.
- **NLB — Network Load Balancer: Layer 4 load balancer** designed for high throughput and ultra-low latency. Handles millions of connections per second using static IPs.
- **GWLb — Gateway Load Balancer:** Routes traffic through third-party **security appliances** like firewalls and IDS/IPS. Uses **GENEVE tunneling** to maintain packet metadata.
- **ASG — Auto Scaling Groups:** Automatically adjusts EC2 instance count based on **scaling policies** or metrics. Ensures performance while optimizing cost through elasticity.

DATABASE SERVICES:

- **Amazon RDS — Relational Database Service:** Managed **SQL databases** (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server) with automated backups and patching. Provides easy scaling and high availability with **Multi-AZ deployments**.
- **Amazon Aurora: Cloud-optimized relational database** compatible with MySQL/PostgreSQL, offering up to **5x–3x performance** over traditional RDS engines. Uses a distributed storage layer for fast failover and high durability.
- **Amazon RDS Proxy:** A **database connection pooling** layer that improves application scalability and reduces DB load. Enhance security by managing credentials with **IAM + Secrets Manager**.
- **Amazon ElastiCache (Redis / Memcached):** Fully managed **in-memory caching service** to accelerate read-heavy or real-time workloads. Reduces database load by serving **sub-millisecond responses**.
- **Amazon MemoryDB:** Durable, **Redis-compatible database** providing ultra-low latency with strong consistency. Ideal for microservices and real-time use cases requiring both speed and durability.

- **Amazon DynamoDB:** Fully managed **NoSQL key-value and document database** with **single-digit ms performance** at any scale. Supports autoscaling, on-demand capacity, and **global multi-Region replication**.
- **DAX — DynamoDB Accelerator: In-memory caching layer** offering microsecond read latency for DynamoDB. Seamlessly integrates with the DynamoDB API, requiring **no code changes**.
- **Amazon Athena:** Serverless **SQL query engine** that analyzes data stored in S3 using standard SQL. No servers to manage, and you only pay per query, ideal for analytics and ad-hoc reporting.

STORAGE:

- **Amazon S3 (Standard, IA, Glacier):** Fully managed **object storage** offering high durability and scalability for any data size. Provides multiple tiers like **Standard**, **Infrequent Access**, and **Glacier** for archival, optimizing cost based on access patterns.
- **Amazon EBS — Elastic Block Store:** Persistent **block storage** for EC2 with AZ-level replication, supporting SSD/HDD performance tiers. Ideal for boot volumes, databases, and workloads requiring consistent low-latency I/O.
- **Amazon EFS — Elastic File System:** Scalable, **multi-AZ NFS file system** that automatically adjusts storage capacity. Supports shared access across many EC2 instances, ideal for web apps and big data workloads.
- **S3 Lifecycle Policies:** Automates **data movement between S3 storage classes** based on age or access frequency. Helps reduce cost by transitioning or expiring objects without manual intervention.
- **AWS Storage Classes:** Tiered storage options like **S3 Standard**, **S3 IA**, **S3 Glacier**, **S3 Glacier Deep Archive**, and **One Zone-IA**. Each offers a trade-off between cost, performance, and availability.
- **AWS Backup:** Centralized backup service for **EBS, RDS, DynamoDB, EFS, FSx, and more**. Enables automated scheduling, retention policies, and compliance reporting across AWS workloads.

CONTAINERIZATION & ORCHESTRATION:

- **Amazon ECS — Elastic Container Service:** Fully managed **container orchestration service** for running Docker containers at scale. Integrates tightly with EC2 and Fargate for flexible compute options.
- **AWS Fargate: Serverless compute** for containers that eliminates the need to manage EC2 instances. Automatically handles scaling, isolation, and resource provisioning for ECS/EKS tasks.

- **Amazon ECR — Elastic Container Registry:** Secure, managed **Docker image repository** with vulnerability scanning and lifecycle policies. Integrates seamlessly with ECS, EKS, and CI/CD pipelines.
- **Amazon EKS — Elastic Kubernetes Service:** Managed **Kubernetes control plane** that simplifies cluster operations. Provides automatic upgrades, scaling, and deep integration with AWS networking and security.
- **AWS Copilot:** CLI tool that streamlines deployment of containerized apps on ECS and Fargate. Handles **infrastructure, CI/CD, and service configuration** using simple commands.
- **AWS App Mesh:** **Service mesh** for microservices providing consistent **traffic control, observability, and security**. Works with ECS, EKS, and on-prem workloads.
- **AWS Cloud Map:** **Service discovery** tool that dynamically registers microservice endpoints. Enables applications to discover resources using **friendly names** instead of IPs.

DEPLOYMENT & INFRASTRUCTURE AS A CODE:

- **AWS CloudFormation:** Infrastructure-as-code service that deploys AWS resources using **declarative templates** (YAML/JSON). Ensures consistent, repeatable environments with **stack-based lifecycle management**.
- **AWS CDK — Cloud Development Kit:** Allows you to define infrastructure using **real programming languages** (TypeScript, Python, Java, etc.). Synthesizes CDK code into CloudFormation templates for easier, modular IaC workflows.
- **AWS Elastic Beanstalk:** Fully managed platform for deploying apps without handling infrastructure. Automatically manages **EC2, load balancers, scaling, and health monitoring** while you focus on application code.
- **AWS AppConfig:** Feature of AWS Systems Manager that lets you **deploy application configurations safely**. Supports versioning, gradual rollout, and validation to prevent config-related failures.

CICD & DevOps tools:

- **AWS CodeCommit (Git repository):** Fully managed **Git repository service** that securely stores source code with encryption and fine-grained IAM access. Supports scalable, private version control for teams.
- **AWS CodePipeline (Orchestration):** **CI/CD orchestration service** that automates build, test, and deployment workflows. Integrates with CodeBuild, CodeDeploy, GitHub, and third-party tools.

- **AWS CodeBuild (Build service):** Managed **build and test service** that compiles code, runs tests, and packages artifacts. Scales automatically and charges only for build time used.
- **AWS CodeDeploy (Deployment automation):** Automates **application deployments** to EC2, on-prem servers, Lambda, and ECS. Supports blue/green and rolling deployments to reduce downtime and risk.
- **AWS CodeStar (DevOps project management):** Provides an integrated **DevOps project dashboard** with CI/CD setup, team permissions, and project templates. Speeds up onboarding and project creation.
- **AWS CodeArtifact (Artifact repository):** Secure, scalable **artifact and package repository** supporting npm, Maven, PyPI, and NuGet. Integrates with CI/CD pipelines and handles dependency versioning.
- **AWS CodeGuru (Code review & profiling):** ML-powered **code analysis and performance profiling** tool. Identifies bugs, inefficiencies, and cost-impacting issues in applications.
- **AWS Cloud9 (Cloud IDE):** Cloud-based **IDE with integrated terminal, debugger, and code editor**. Allows collaborative development directly in the browser with AWS credentials preconfigured.

INTEGRATION & MESSAGING:

- **Amazon SQS: Simple Queue Service:** Fully managed **message queuing** service enabling decoupling between distributed systems. Supports **standard and FIFO queues** for reliable, scalable message processing.
- **Amazon SNS: Simple Notification Service:** **Pub/Sub messaging** service for sending notifications to multiple subscribers. Supports delivery to email, SMS, Lambda, SQS, and HTTP endpoints with high fan-out performance.
- **Amazon Kinesis — Data Streams:** Real-time **streaming data ingestion** service handling massive throughput. Allows consumers to process data with sub-second latency for analytics and event processing.
- **Amazon Kinesis — Data Firehose:** Fully managed **data delivery service** that loads streaming data into S3, Redshift, OpenSearch, and third-party tools. Automatically scales and performs optional data transformation.
- **Amazon Kinesis — Data Analytics:** Serverless engine that runs **SQL or Apache Flink** to process streaming data in real time. Enables real-time dashboards, anomaly detection, and ETL on streams.
- **Amazon Kinesis — KCL (Kinesis Client Library):** Client-side library that simplifies building **sharded data consumers** for Kinesis Streams. Handles checkpointing, load balancing, and fault recovery automatically.

- **Amazon EventBridge:** Serverless **event bus** for routing events between AWS services, SaaS apps, and custom apps. Supports rule-based filtering, schema registry, and event-driven architectures.

NETWORKING & DNS:

- **Route 53 (DNS & routing):** Highly available **DNS and domain management** service supporting routing policies like latency, weighted, and geolocation. Integrates with health checks for failover-based traffic routing.
- **AWS CloudFront:** Global **CDN service** that caches content at edge locations for low-latency delivery. Supports HTTPS, signed URLs, and integrates tightly with S3, ALB, and Shield.
- **VPC: Virtual Private Cloud:** Isolated **virtual network** where you define subnets, routing, and security controls. Enables granular control of networking for EC2, Lambda, containers, and databases.
- **Security Groups: Stateful firewalls** controlling inbound/outbound traffic at the instance or ENI level. Automatically allow return traffic and operate as virtual "allow lists."
- **NACLs — Network ACLs: Stateless subnet-level firewalls** that evaluate inbound and outbound rules independently. Useful for setting broader **allow/deny** network boundaries.
- **VPC Peering / Transit Gateway:** Peering enables **direct communication** between two VPCs, while Transit Gateway centralizes routing for **hub-and-spoke multi-VPC** architectures. Both simplify internal connectivity at scale.
- **Direct Connect:** Dedicated **high-bandwidth, low-latency link** between on-prem data centers and AWS. Improves hybrid workloads by offering predictable network performance and reduced data transfer costs.

SERVERLESS SERVICES:

- **AWS Lambda:** Serverless **compute service** that runs functions in response to events without managing servers. Scales automatically and charges only for execution time.
- **Amazon DynamoDB:** Fully managed **NoSQL key-value/document database** offering single-digit millisecond performance. Supports autoscaling, streams, and global replication for serverless workloads.
- **Amazon API Gateway:** Managed service for creating **REST, WebSocket, and HTTP APIs** with built-in authentication, throttling, and caching. Integrates seamlessly with Lambda and other backend services.
- **AWS SAM (Serverless Application Model):** Infrastructure-as-code framework for defining **serverless applications** using simplified syntax. Supports local testing, packaging, and deployment through CloudFormation.

- **AWS Step Functions:** Orchestration service for building **stateful workflows** that coordinate Lambda functions and AWS services. Supports visual state machines, retries, error handling, and parallel execution.
- **AWS AppSync:** Managed **GraphQL API service** that connects to DynamoDB, Lambda, RDS, and more. Provides real-time data via subscriptions and simplifies API development.
- **AWS Amplify:** Framework and tooling for building **frontend + backend cloud applications** with authentication, APIs, hosting, and CI/CD. Integrates tightly with mobile and web frameworks.

SECURITY & ENCRYPTION:

- **AWS KMS: Key Management Service:** Managed service for creating and controlling **encryption keys (KMS CMKs)**. Integrates with most AWS services for **at-rest and in-transit encryption** with detailed access controls.
- **AWS Secrets Manager:** Securely stores **API keys, passwords, tokens**, and rotates them automatically. Provides fine-grained IAM access and seamless integration with RDS and Lambda.
- **SSM Parameter Store:** Centralized storage for **configuration values and secrets**, supporting hierarchies and versioning. Offers standard and advanced tiers with optional KMS encryption.
- **AWS CloudHSM:** Dedicated hardware appliances offering **FIPS 140-2 Level 3 compliant** key management. Ideal for workloads needing **full control over HSMs** for compliance requirements.
- **AWS WAF / Shield:** WAF protects web apps using **custom rules** against SQLi, XSS, and bot attacks; Shield provides **DDoS protection** (Standard + Advanced) for CloudFront, ALB, and Route 53.
- **AWS Certificate Manager (ACM):** Automates provisioning and renewal of **SSL/TLS certificates** for AWS services. Simplifies HTTPS with managed certificates for ALB, CloudFront, and API Gateway.
- **AWS Private Certificate Authority (CA):** Managed **private CA** service for issuing internal TLS certificates. Simplifies PKI operations without maintaining your own CA infrastructure.
- **AWS EC2 Nitro Enclave:** Creates isolated, lightweight, **secure compute environments** for handling sensitive data. Provides cryptographic attestation and prevents external network or storage access.

MISCELLANEOUS/ SPECIALIZED SERVICES:

- **Amazon SES (Simple Email Service):** Fully managed **email sending and receiving** service for transactional and marketing messages. Supports DKIM, SPF, dedicated IPs, and scalable delivery with high deliverability rates.
- **Amazon OpenSearch:** Distributed **search, analytics, and logging engine** used for full-text search and observability dashboards. Provides managed clusters, ingestion pipelines, and integrations with CloudWatch and Kinesis.
- **Amazon MSK (Managed Kafka):** Fully managed **Apache Kafka** service handling cluster provisioning, scaling, and patching. Ensures secure, durable streaming pipelines without managing brokers manually.
- **AWS FIS: Fault Injection Simulator:** Chaos engineering service for running **controlled fault experiments** on AWS workloads. Helps validate resilience by simulating latency, failures, and resource disruptions.
- **Amazon EC2 Systems Manager:** Unified operations tool for **patching, automation, inventory, and remote command execution**. Works across EC2, hybrid servers, and on-prem systems with centralized control.
- **AWS Cloud Map (Service discovery):** Managed **service registry** that stores names and endpoints for microservices. Enables applications to discover services dynamically via API calls or DNS.
- **AWS Step Functions:** Serverless **workflow orchestration** using state machines to coordinate AWS services. Supports retries, branching, parallel execution, and long-running processes.
- **AWS AppConfig:** Configuration management service for **safe, gradual config rollouts** across applications. Includes validation, versioning, and automated rollback to avoid config-based failures.

ARTIFICIAL INTELLIGENCE:

- Resources are created in an Azure subscription.
- Resource Group: Collection of Azure resources. Resources are created within a resource group.
- Azure AI service resource is created on Azure portal. It has - 1. Endpoint URI, 2. Subscription key, 3. Location
- Multi service resource: Same key and endpoint for all resources, Single service resource: Unique key and endpoint per resource.
- Resources are accessed using REST API, SDK, and portal (studio). Key and endpoint are required in authentication header while using REST API.
- Development: Training: Feature - Input, Label - Output. Inferencing: Predicting the label. Confidence scores - Probability of prediction.
- Security: Keys can be stored in Azure key vault and control access using RBAC. Regenerate keys as a best practice.
- Monitoring: Create alert rules, view metrics, enable diagnostic logging, monitor cost & performance
- Deployment: Applications can run on Docker, Azure Container Instance etc. Subscription key not required for containerized applications.

COMPUTER VISION:

- **Optical Character Recognition (OCR):** Used for detecting the words, sentences and paragraphs in an image. Uses **Read API**. Service: **Azure AI vision**.
- **Image Analysis:** Used for detecting objects, faces, or text in an image. It can describe the image, generate tags and thumbnail, do background removal, get image metadata, do color and type analysis. Service: **Azure AI vision**.
- **Image Classification:** Used to classify a given image into one or multiple categories. Image classification can be multiclass (Image belongs to one label) or multilabel (Image belongs to multiple labels). Service: **Azure AI Custom Vision**.
- **Object detection:** Used to detect the presence and location of objects in the image. This is done by bounding box co-ordinates: left, top, width, height. Top left corner of bounding box and width and height of box. Service: **Azure AI Custom Vision**.
- Image classification and object detection can be done by **training custom image models** provided by Azure. So, we upload labeled images and train the model and check its accuracy. If we are not satisfied, we train it again until we get correct results.
- **Face analysis:** Used to understand different facial features of a person. Recognizing a particular person, comparing two face images, detecting faces in an image etc. Used a GUID assigned to every face. Service: **Azure AI Face, Face API**.

- **Video Indexing:** Used to analyze video content. Content moderation, scene segmentation, speech transcription, face recognition, OCR, topic, sentiment understanding in a video.
Service: **Azure Video Indexer API.**

NLP – TEXT

Pre-configured features: No model labeling or training. Service: **Azure AI Language**

- Text analysis
- Text summarization
- Text translation
- Language detection - AnalyzeText API, Azure AI Language detection API
- Personally identifiable information (PII) detection
- Content moderation
- Entity linking
- Named entity recognition - Person, location, company, datetime, quantity, etc.
- Sentiment analysis and opinion mining
- Key phrase extraction

Learned features: Label data, train, evaluate and deploy the model

- Conversational language understanding (CLU)
- Question answering
- Custom named entity recognition
- Custom text classification – Single/ multi label classification

NLP – SPEECH:

- **Speech synthesis:** The ability to generate spoken output. Used to convert text to speech. Input is text, and output is the speech which is delivered to a speaker or any audio file.
- Different voices can be generated using **Speech Synthesis Markup Language (SSML)**
Service: **Text to speech API, Batch synthesis API.**
- **Speech recognition:** The ability to detect and interpret spoken input. Used to convert speech to text. Input is audio file or given to a microphone, and output is the text.
- Real-time transcription or batch transcription can be done. Service: **Speech to text API.**
- **Language translation:** Used to detect language, translate, and transliterate the given input.
- It can be used for **profanity filtering, sentiment analysis**, and recognizing **key phrases** and **entities**. Service: **Azure AI Translator, Text Analytics API**

QUESTION ANSWERING:

- A **set of question/answer pairs** is stored in a **knowledge base**.
- It uses **Azure AI Search** or Cognitive Services to **extract precise answers** from documents such as predefined knowledge base or data sources.
- Can combine **retrieval with generative models (RAG)** to produce **accurate, grounded answers**.
- Supports **multi-turn Q&A**, allowing the model to **use previous context** for better responses.
Service: **Azure AI Language, Azure AI Bot Service**

CONVERSATIONAL LANGUAGE UNDERSTANDING:

- Identify **entities** and **intent** from a person's **utterance**.
- Entity is an object, person, organization, location, etc. Intent is the action to be taken with the entity. Utterance is simply a statement about what a person says.
- There can be learned, list, and prebuilt entities.
- Intent recognition
- Entity extraction
- Dialog management
- Personalization
- Multilanguage support
- Multiturn conversation: Model needs more input from user. So there needs to be a back and forth between user and model. For this, the model gives an answer and a follow up prompt or question.

DOCUMENT INTELLIGENCE:

- Azure AI Document Intelligence uses **prebuilt, custom, and composite models** to extract structured data from documents.
- **Prebuilt models** (invoice, receipt, ID, business card, W2, general document, etc.) provide instant extraction of text, tables, fields, and key-value pairs.
- **General Document Analysis** includes the **Read, General Document, and Layout** models for OCR, structure, and semantic extraction.
- All models return results in **JSON** with text, entities, selection marks, tables, and fields.
- **Custom models** are trained when prebuilt models don't meet requirements; more training samples → higher accuracy.

- **Composed models** combine multiple custom models to automatically classify and analyze different form types.
- **Custom model types** include **template models** (fixed layout) and **neural models** (any structure, including unstructured).
- Document Intelligence integrates with **Azure AI Search** using **custom skills** to enrich search indexes with extracted fields.
- Custom skills can be implemented using **Azure Machine Learning** (ML models) or **Custom Web API** endpoints.
- Standard tier allows **support-request-based quota increases**; Free tier cannot be used with multi-service resources.
- Service: Azure Document Intelligence resource

KNOWLEDGE MINING:

- **Azure AI Search (PaaS)**: Indexes and searches structured, semi-structured, and unstructured data using **Apache Lucene** under the hood.
- **Knowledge Mining**: *Ingest* → *Enrich* → *Explore* using data sources like Blob Storage, SQL DB, Cosmos DB, etc.
- **Indexer**: pulls data, converts it to JSON, and loads it into a **Search Index**; or you can push JSON directly via REST/SDK.
- **Search Index**: It holds JSON documents defined by a schema with searchable, filterable, and sortable fields.
- **Enrichment**: It uses built-in NLP/image skills or custom skills to transform data during indexing.
- **Knowledge Store** saves enriched outputs such as **object, file, or table projections**.
- Querying supports **simple, full Lucene**, filters, sort, scoring, boosting, analyzers, and multi-language support.
- **Scalability** uses replicas (availability), partitions (storage/distribution), and search units = replicas × partitions.
- **Semantic search**: improves relevance using NLP (BM25 + semantic ranking) and provides captions/answers (max 50 results).
- **Vector search**: stores and retrieves embeddings for RAG, similarity search, and multimodal scenarios.
- Service: **Azure AI Search, Azure Cognitive Search**

GENERATIVE AI:

- AI which generates content such as text, image, audio, video, or code based on patterns learned from data.
- LLMs work by getting trained on massive information and generating content which is similar to the data the model was trained on. Making the content more human like.
- Prompt engineering: Technique of giving precise inputs or prompts to LLM to produce accurate, reliable answers
- Techniques: Prompt-engineering techniques include zero-shot (no examples), few-shot (guided examples), chain-of-thought (stepwise reasoning), role prompting (assigned persona), instruction prompting (clear tasks), contextual prompting (added background), constraint prompting (rules/format limits), and multi-step prompting (sequential guidance).
- RAG: A technique where LLM is connected to an external, real time data source, outside its training data, to generate more relevant and accurate response and reduce hallucination.
- Service: **Azure Open AI**

AGENTIC AI:

- **Agentic AI** refers to AI systems that can autonomously plan, reason, and take actions toward goals with very little or no human intervention.
- They use **long-term memory, planning, and tool use** (APIs, databases, external systems) to complete complex tasks.
- These agents operate in **loops**: observe → reason → act → reflect → improve.
- They can **break down tasks**, call other services or models, and adapt their actions based on feedback.
- Agentic AI enables **end-to-end workflows**, from decision-making to execution, making applications more dynamic and intelligent.

AZURE AI FOUNDRY:

- Azure AI Foundry is Microsoft's platform for **building, customizing, deploying, and managing generative AI and machine-learning solutions**.
- Offers built-in tools like model catalogs, prompt engineering, RAG components, evaluations, and integrated MLOps.
- Provides a unified workspace to experiment with models, datasets, prompts, and pipelines.
- Supports end-to-end AI lifecycle management, including monitoring, safety, and compliance controls.

- Integrates seamlessly with Azure services (OpenAI, AI Search, Kubernetes, Storage) for scalable production deployment.
- **Prompt Flow** is a development framework for defining flows that orchestrate interactions with an LLM.

RESPONSIBLE AI:

- **Responsible AI** is a set of principles, practices, and governance frameworks that ensure **AI systems are designed, developed, deployed, and used in ways that are safe, ethical, trustworthy, and aligned with human values.**

Key principles

- **Fairness** – AI should treat all people equitably and avoid bias.
- **Reliability & Safety** – AI systems should function as intended and be safe under all expected conditions.
- **Privacy & Security** – Protect user data and secure AI systems against misuse or attacks.
- **Inclusiveness** – AI should empower and be usable by as many people as possible.
- **Transparency** – Users should understand how and why AI makes decisions.
- **Accountability** – Organizations should be responsible for the outcomes of their AI systems.

MISCELLANEOUS:

- AI => ML => NN => DL => LLM. And DS.
- Generative models: Translation, text summarization, and 2. Discriminative models: Sentiment analysis, image recognition, and 3. Language models: Entity recognition, Question answering
- Supervised learning: 80/20 - 80% training data and 20% evaluation
 - Regression - Model predicts a numeric value
 - Classification: Binary - Only two categories: binary true/false or positive/negative prediction
 - Multiclass: Multiple categories of labels.
 - Multilabel classification: An item may belong to multiple labels.
- Unsupervised learning: cluster unlabeled datasets (subsets called clusters)
 - Clustering: Grouping the items into clusters by observing similarities in the features.